

Algorithmes et Systèmes Distribués — Fiche de révision

M1 Informatique (ILIA) — d'après le cours d'E. Fabiani, UBO

1. Qu'est-ce qu'un système distribué ?

Système distribué : ensemble de processus autonomes, d'importance égale (pas de centralisation fixe), qui coopèrent via des échanges de messages explicites (pas de mémoire partagée) sur un réseau de communication, pour atteindre un objectif commun.

Architecture distribuée : nœuds de calcul matériels + dispositif de communication matériel (NOC, câble, radio...).

Système distribué (logiciel) : ensemble de processus + services de communication au-dessus de l'architecture.

Algorithme distribué : algorithme découpé en processus communicants, exécuté sur l'architecture, en utilisant les services du système distribué.

Distribué vs concurrent vs parallèle

Type	Caractéristique principale
Concurrent	Processus en compétition pour une ressource (mémoire centrale partagée), entrelacement d'exécution.
Parallèle	Ressources matérielles multiples pour accélérer une même application (objectif : performance/efficacité).
Distribué	Processus non nécessairement en concurrence, mémoire privée par processus, communication par messages ; objectif pas forcément la performance (ex : routage réseau).

2. Objectifs et problèmes des systèmes distribués

Objectifs

- Performance : répartition des tâches, parcimonie des messages.
- Extensibilité (scalabilité) : ajouter un processus ne doit pas nécessiter de reprogrammer les autres, ni dégrader les performances.
- Fiabilité : la défaillance d'un processus/canal ne doit pas faire tomber tout le système (citation Lamport).

Problèmes à résoudre

- Distribution des tâches (rôles, homogène/hétérogène, tolérance aux pannes).
- Répartition mémoire et acheminement : unicast / multicast / broadcast.
- Résolution des accès concurrents aux ressources partagées.
- Fiabilisation des communications : pertes, délais variables/non bornés, interruptions, ordre aléatoire d'arrivée.
- Éviter les interblocages : Deadlock (tous bloqués en attente d'un événement qui n'arrivera jamais) et Livelock (tous actifs mais bloqués dans un cycle d'états, ex. drones qui montent/descendent en boucle).
- Synchronisation (même état d'avancement) et Terminaison (détecter la fin, état stable partagé par tous).
- Observation du système : pas d'état global instantané connaissable ; observer peut modifier le comportement (effet de bord).

Problématique de l'ordre

- Ordre total : possible en séquentiel (trace des instructions).
- Ordre partiel : dans un système distribué, ordre local par processus + synchronisations $\Rightarrow$ ordre partiel entre ensembles d'états ; l'ordre observé peut varier d'une exécution à l'autre mais doit rester compatible avec les synchronisations/messages échangés.

Correction d'un algorithme distribué

- Sûreté : résultats conformes + absence de deadlock.
- Vivacité : progression garantie (pas de livelock) + terminaison.
- Validation difficile : non-déterminisme des entrelacements $\Rightarrow$ bugs non reproductibles.
- Méthodes : preuve formelle (CSP), model checking, correction par construction (patrons de conception).

3. Modèles de communication

Modes de connexion

- Mémoire partagée / Communication point à point / Diffusion (broadcast, multicast).

Modèle temporel des messages

Mode	Principe	Avantage(s)	Inconvénient(s)
Synchrone (rendez-vous)	Émetteur et récepteur présents en même temps ; émetteur bloqué jusqu'à réception ; canal bloquant de contenance nulle.	États connus des deux côtés ; sert aussi de synchronisation ; acquittement implicite.	Fort couplage temporel.
Asynchrone	Émetteur n'attend pas ; tampons (producteur/consommateur) ; réception bloquante ou non.	Indépendance temporelle des correspondants.	Pas d'acquittement implicite, pas de lien entre états, gestion d'erreurs plus dure.
Invocation à distance	Appel de procédure à distance avec paramètres, attente du résultat (rendez-vous étendu).	Modèle proche de la programmation classique.	Couplage fort, latence réseau.

Désignation et appel

- Directe (nom du processus) vs indirecte (nom d'un canal/port).
- Symétrique (émetteur et récepteur se nomment) vs asymétrique (récepteur attend sans nommer de source — modèle client-serveur).

Trois types d'interaction entre processus

- Synchronisation : contrôler l'évolution correcte, bloquer les processus « en avance ».
- Coopération : transmission de valeurs nécessaires à la tâche des autres.
- Compétition : accès concurrent à une ressource, verrouillage.

Problèmes classiques de synchronisation/coopération

Problème	Définition courte
Exclusion mutuelle	Une seule ressource, un seul processus à la fois.
Cohorte	Exclusion mutuelle généralisée à un groupe de taille max donnée.
Élection d'un leader	Un seul processus élu ; lui seul sait qu'il est leader, les autres savent qu'ils ne le sont pas.
Consensus	Vote oui/non ; action acceptée ssi tous ont voté oui.
Rendez-vous	Tous les processus bloqués jusqu'à ce que tous soient prêts.
Producteurs-consommateurs	Tampon borné ; écriture bloquante si plein, lecture bloquante si vide ; chaque message consommé une seule fois.
Lecteurs-rédacteurs	Un seul rédacteur à la fois, plusieurs lecteurs en parallèle.

4. Le modèle synchrone

Modélisation du réseau

- Réseau de processus = graphe orienté ; nœuds = processus, arcs = canaux unidirectionnels contenant au plus 1 message.
- Distance(i,j) = longueur du plus court chemin (en sauts) ; Diamètre = distance maximale sur toutes les paires — borne le nombre d'échanges nécessaires pour toute communication (utile pour diffusion et terminaison).
- Voisin entrant (canal de j vers i) / voisin sortant (canal de i vers j).

Principe

- On synchronise les itérations via des communications bloquantes logicielles : aucun processus n'a plus d'un tour d'avance.
- Communications déterministes logicielles : chaque processus sait qu'il enverra/recevra un message à chaque voisin à chaque tour.
- Par construction : pas de deadlock (tout envoi a une réception, et inversement).
- Attention : le comportement INTERNE des processus reste asynchrone (chacun exécute à son rythme entre deux points de synchro) ; les canaux physiques peuvent rester asynchrones si on ajoute logiquement acquittement + attente + retransmission.

- Jamais plus d'un message échangé à la fois entre deux processus voisins.

Message null

- Si un processus n'a rien d'utile à envoyer à un tour, il envoie un message null (valeur particulière).
- Un message null reçu est ignoré par l'algorithme applicatif.
- En pratique : peut être implémenté par timeout ; sert aussi à simuler des pannes (perte de message = transformation aléatoire d'un message utile en null).

Mise en œuvre (3 étapes par tour)

- 1. Envoi des messages à tous les voisins sortants + réception des messages de tous les voisins entrants.
- 2. Modification de l'état courant en fonction des messages reçus (fonction trans).
- 3. Génération des messages à envoyer au tour suivant (fonction msgs).
- Formalisation : $\text{in}(i)/\text{out}(i)$ (E/S), $\text{states}(i)$ (états, éventuellement infini), $\text{start}(i)$ (état initial), $\text{trans}(i) : \text{states} \times \text{messages} \rightarrow \text{states}$, $\text{msgs}(i) : \text{states} \times \text{out}(i) \rightarrow M \cup \{\text{null}\}$. État terminal : reste constant, ne produit plus de messages.

En Go (CSP)

- Séquence logique par tour : C (calcul + génération messages), E (envoi), R (réception) $\rightarrow C, E, R, C, E, R, \dots$
- La séquence C, E, R provoquerait un interblocage (rendez-vous CSP) $\Rightarrow$ on parallélise E et R (et les accès aux tableaux de canaux, en évitant les dépendances croisées).

5. Algorithmes distribués synchrones — à connaître

Élection de leader sur anneau unidirectionnel (synchrone)

- Contexte : anneau, id unique par nœud, nombre de nœuds inconnu.
- Init : message sortant $:= \text{id}$; état $:= \text{non-leader}$.
- À chaque tour, v = message reçu :
 - $v = \text{null}$ $\rightarrow$ renvoyer null
 - $v > \text{id}$ $\rightarrow$ retransmettre v (v continue son tour de l'anneau)
 - $v < \text{id}$ $\rightarrow$ ne rien retransmettre (null)
 - $v = \text{id}$ $\rightarrow$ l'identifiant est revenu : ce nœud devient leader
- Terminaison en $N+1$ tours (N = diamètre = nombre de nœuds de l'anneau).
- Variante : un 2^e tour complet permet au leader de se faire connaître de tous, ou de lancer des calculs globaux (max, sommes, diamètre, terminaison).

Parcours en largeur BFS (synchrone)

- Contexte : topologie quelconque, réseau fortement connecté, diamètre inconnu, un nœud source root.
- Init : $\text{root.visité} = \text{VRAI}$ (envoie search à ses voisins) ; les autres $\text{visité} = \text{FAUX}$ (envoient null).
- À chaque tour, si $\text{visité} = \text{FAUX}$ et qu'un message reçu = search : visité devient VRAI, le canal d'origine devient le "père", et on retransmet search au tour suivant. Sinon on envoie null.
- Résultat : arbre de recouvrement (chaque processus connaît son père) ; terminaison en N tours (N = diamètre).
- Utilité du BFS : élection de leader même sans connaître le diamètre (chacun démarre un BFS, calcule le max des id via l'arbre, compare à son propre id) ; calcul du diamètre (BFS concurrents + diffusion/max des profondeurs) ; diffusion de valeurs et remontée de résultats si liens bidirectionnels.

6. Le modèle asynchrone

Motivations

- Le modèle synchrone vide les canaux à chaque tour (visibilité locale) mais impose des communications fréquentes ; le plus lent fixe le rythme de tous.
- L'asynchrone est plus proche de la réalité des réseaux : émission libre, canaux avec capacité de stockage, émission/réception asynchrones par rapport au processus, pas de borne sur le temps de transmission.
- Tamponnage : en sortie (medium indisponible, concurrence), en entrée (processus occupé/bloqué/préempté), dans le réseau (files locales), et même dans le medium physique (ex. câble transocéanique = des milliers de paquets "stockés" en transit).

Canal asynchrone C(i,j)

- Opérations découplées : $\text{send}(i)(m)$ et $\text{receive}(j)(m)$.
- Modélisation : le canal est lui-même un processus contenant une FIFO (tableau + indices read/write).
- $\text{send}(i)(m)$: communication bloquante côté émetteur. $\text{receive}(i)(m)$ = requête (reçue par le canal) + réception effective (envoyée par le canal).
- Peut être doté de mémoire (FIFO ou autre) ; pertes = effacement de messages ; réordonnancement = permutations.

En Go

- Le canal est une goroutine avec `select` sur `sendChan` (écriture FIFO) et `rcvReqChan/rcvDataChan` (lecture FIFO), avec gestion des cas file vide / pleine.

7. Algorithme distribué asynchrone — exemple

Élection de leader sur anneau asynchrone

- Canaux directionnels asynchrones, chaque processus a un id unique qu'il émet sur son canal sortant.
- À réception de `rcvID` : si $\text{rcvID} > \text{id}$ → on réémet `rcvID` ; si $\text{rcvID} < \text{id}$ → on ne fait rien ; si $\text{rcvID} = \text{id}$ → on devient leader.
- Différence avec le cas synchrone : les messages peuvent s'accumuler en nombre arbitraire dans les files des nœuds (pas de notion de "tour" commun).

8. Synthèse : synchrone vs asynchrone

	Modèle synchrone	Modèle asynchrone
Rythme	Par tours, tous les nœuds progressent ensemble (décalage ≤ 1)	Chaque processus à son rythme, pas de notion de tour global
Canal	Contenance ~nulle, vidé à chaque tour	FIFO avec capacité de stockage
Deadlock	Impossible par construction	À gérer explicitement
Terminaison typique	Bornée par le diamètre (N ou N+1 tours)	Non bornée a priori
Message spécial	Message null (absence de message utile)	Pas d'équivalent direct